

LAPORAN TEKNIS KOMPREHENSIF

DiaLens - Your AI Diabetes Risk Screener



Disusun Oleh:

Djibrani Yuda Febriansyah	CACC299D6Y2632
Jonathan Alfa Grace Julianto	CACC463D6Y2610
Farih Kamil Zulfah	CDCC299D6Y2755
Reza Maulana	CDCC299D6Y2532
Abinaya Azhar Probo Kusomo	CFCC198D6Y1830
Edwin Fadhilah Putra	CFCC198D6Y1887

CODING CAMP 2026 POWERED BY DBS FOUNDATION

2026

ABSTRAK

Prevalensi Diabetes Melitus (DM) di Indonesia menunjukkan tren peningkatan yang signifikan, namun tingkat diagnosis awal masih sangat rendah akibat berbagai hambatan akses pada metode skrining konvensional. Keterlambatan diagnosis ini berujung pada komplikasi kesehatan yang fatal dan pembengkakan beban finansial. Untuk mengatasi permasalahan tersebut, proyek ini mengembangkan **DiaLens**, sebuah platform aplikasi web skrining risiko diabetes dini yang inklusif, cerdas, dan interaktif untuk memfasilitasi deteksi mandiri oleh masyarakat tanpa bergantung pada uji laboratorium awal.

Pengembangan perangkat lunak ini mengadopsi metodologi hibrida CRISP-DM dan *Software Development Life Cycle* (SDLC) yang direalisasikan ke dalam arsitektur terdistribusi (*3-Tier Microservices*). Mesin inferensi kecerdasan buatan dibangun menggunakan kerangka kerja FastAPI, menjalankan arsitektur *Deep Learning* kustom berbasis *Model Subclassing* yang dilatih menggunakan himpunan data klinis BRFSS. Lapisan orkestrasi logika bisnis dan manajemen pangkalan data dikelola melalui *API Gateway* berbasis Express.js dan MongoDB, sementara antarmuka interaktif di sisi klien dikembangkan menggunakan Next.js.

Hasil evaluasi akhir menunjukkan bahwa model memiliki keandalan tinggi sebagai instrumen skrining medis dengan capaian metrik *Recall* sebesar 86%. Performa ini diraih melalui penerapan *Focal Loss* dan penyetelan ambang batas (*threshold tuning*) sebesar 0,45 guna meminimalkan rasio *False Negative* secara drastis. Lebih jauh, sistem secara efektif menjembatani permasalahan "kotak hitam" algoritma melalui integrasi *Explainable AI* (SHAP) dan kecerdasan buatan generatif (Claude API). Pendekatan ini memungkinkan DiaLens untuk tidak sekadar menampilkan probabilitas numerik, melainkan mentransformasikan kontribusi fitur pasien menjadi rekomendasi gaya hidup yang dipersonalisasi dan dapat ditindaklanjuti (*actionable*). Secara keseluruhan, DiaLens terbukti menjadi instrumen preventif yang tangguh, efisien, dan transparan dalam mendukung upaya deteksi dini serta edukasi risiko penyakit metabolik di masyarakat.

Kata Kunci: Diabetes Melitus, Deteksi Dini, *Deep Learning*, *Microservices*, *Explainable AI*, SHAP, Aplikasi Web.

BAB I

PENDAHULUAN DAN PROBLEM DISCOVERY

1.1 Latar Belakang Masalah

Diabetes Melitus (DM) diklasifikasikan sebagai penyakit metabolik kronis yang dipicu oleh disfungsi sekresi maupun resistensi insulin, yang berujung pada lonjakan kadar glukosa darah (Hanim et al., 2026). Karakteristik penyakit ini yang kerap berkembang secara senyap tanpa keluhan spesifik di fase awal membuat sebagian besar penderita tidak menyadari kondisi kesehatannya.

Di Indonesia, urgensi penanganan DM semakin mendesak. Berdasarkan temuan Wulandari et al. (2026), angka prevalensi nasional terus merangkak naik dari 10,9% pada 2018 menjadi 11,7% di tahun 2023. Ironisnya, tren peningkatan ini tidak dibarengi dengan tingkat kesadaran deteksi, di mana rasio kasus yang berhasil didiagnosis dan mendapat penanganan medis justru mandek di kisaran 2%.

Kesenjangan yang lebar antara tingginya prevalensi dan rendahnya tingkat diagnosis ini memicu efek domino yang fatal. Keterlambatan intervensi medis membuka jalan bagi komplikasi mematikan pada organ vital seperti jantung, ginjal, saraf, dan mata. Selain menurunkan kualitas hidup penderita secara drastis, kondisi ini juga menguras sumber daya ekonomi. Wulandari et al. (2026) mencatat bahwa beban finansial untuk mengelola pasien DM beserta komplikasinya bisa membengkak hingga tiga kali lipat dibandingkan penanganan kasus tanpa komplikasi.

Di sisi lain, upaya pencegahan primer melalui skrining konvensional masih terbentur realitas operasional di lapangan. Hanim et al. (2026) mengungkapkan bahwa kendala biaya, keterbatasan waktu, serta ketimpangan akses menuju fasilitas kesehatan menjadi biang keladi rendahnya partisipasi masyarakat. Menjawab tantangan tersebut, inovasi teknologi digital sangat dibutuhkan untuk menghadirkan alternatif skrining mandiri yang praktis, inklusif, dan efisien, sehingga masyarakat dapat mengenali profil risiko mereka lebih awal sebelum timbul komplikasi klinis.

1.2 Rumusan Masalah

Berdasarkan pada uraian latar belakang tersebut, rumusan masalah dalam pengembangan proyek ini difokuskan pada:

1. Bagaimana merancang antarmuka aplikasi web yang intuitif dan ramah pengguna (*user-friendly*), sehingga masyarakat lintas generasi dapat melakukan deteksi dini secara mandiri tanpa bergantung pada uji laboratorium awal?

2. Bagaimana merancang bangun serta mengintegrasikan algoritma model prediktif ke dalam sistem untuk mengklasifikasikan tingkat risiko diabetes pengguna berdasarkan input variabel gaya hidup dan metrik kesehatan?

1.3 Tujuan Proyek

Selaras dengan rumusan masalah di atas, proyek ini dikembangkan dengan sasaran utama:

1. Menghadirkan platform digital inklusif yang mampu memangkas hambatan akses masyarakat terhadap fasilitas skrining awal deteksi risiko Diabetes Melitus.
2. Menciptakan ekosistem aplikasi ganda yang tidak terbatas pada deteksi risiko, tetapi juga berperan sebagai medium edukasi preventif guna meningkatkan literasi kesehatan masyarakat akan bahaya penyakit metabolik.
3. Mengimplementasikan model prediktif cerdas yang berkapasitas memproses input pengguna secara dinamis untuk menghasilkan klasifikasi risiko kesehatan yang efisien dan *real-time*.

1.4 Batasan Masalah

Agar pengembangan proyek ini tetap fokus dan terarah pada sasaran yang telah ditentukan, ruang lingkup dan batasan masalah ditetapkan sebagai berikut:

1. Sistem yang dirancang murni berfungsi sebagai instrumen skrining atau alat bantu deteksi dini. Aplikasi ini sama sekali tidak diperuntukkan sebagai pengganti asesmen dokter maupun dijadikan rujukan diagnosis medis yang bersifat definitif.
2. Proses komputasi risiko sepenuhnya bergantung pada mekanisme pelaporan mandiri (*self-reported*). Pengguna bertanggung jawab secara langsung untuk memasukkan data profil kesehatan dan gaya hidup mereka melalui formulir terstruktur yang disediakan di dalam sistem.
3. Solusi interaktif ini diimplementasikan dalam bentuk aplikasi berbasis web yang diintegrasikan dengan algoritma *Machine Learning*. Kapabilitas prediktif dari sistem tersebut dibatasi secara eksklusif pada pola yang telah dipelajari dari dataset klinis spesifik pada fase pelatihan sebelumnya.

BAB II

METODOLOGI DAN DOKUMENTASI TEKNIS

2.1 Deskripsi Dataset

Pengembangan sistem deteksi dini ini ditunjang oleh pemrosesan data sekunder publik yang diekstraksi dari Diabetes Health Indicators Dataset melalui repositori Kaggle. Secara fundamental, himpunan data ini merupakan representasi komprehensif dari hasil survei *Behavioral Risk Factor Surveillance System* (BRFSS) tahun 2015.

Struktur data pada studi ini memuat variabel target (label) yang memetakan probabilitas individu ke dalam tiga klasifikasi kesehatan: sehat, prediabetes, dan diabetes. Guna membangun fondasi prediksi yang kuat, sistem memproses sejumlah variabel prediktor (*features*) yang dikategorikan ke dalam tiga dimensi utama:

1. Dimensi Klinis (Kondisi Medis)

Mencakup riwayat tekanan darah tinggi (*HighBP*), tingkat kolesterol (*HighChol*), Indeks Massa Tubuh (BMI), serta rekam jejak penyakit kardiovaskular (*HeartDiseaseorAttack*) dan stroke.

2. Dimensi Perilaku dan Gaya Hidup

Memetakan tingkat aktivitas fisik harian (*PhysActivity*), asupan nutrisi berupa buah (*Fruits*) dan sayur (*Veggies*), serta paparan zat adiktif seperti rutinitas merokok (*Smoker*) dan konsumsi alkohol berlebih (*HvyAlcoholConsump*).

3. Dimensi Demografi dan Kualitas Hidup

Meliputi usia (*Age*), jenis kelamin (*Sex*), evaluasi kesehatan secara umum (*GenHlth*), stabilitas kesehatan mental (*MentHlth*), rekam hari dengan kondisi fisik buruk (*PhysHlth*), hingga gangguan mobilitas fisik (*DiffWalk*).

2.2 Alat dan Teknologi

Pengembangan proyek ini dikerjakan oleh tim fungsional yang terdiri dari Data Scientist, AI Engineer, dan Full-Stack Developer. Mengingat kolaborasi ini menuntut ritme kerja yang sangat dinamis, manajemen versi dan integrasi basis kode dikendalikan secara tersentralisasi melalui GitHub.

Di dalam struktur organisasi proyek ini, pembagian ruang lingkup dan tanggung jawab arsitektural diatur sebagai berikut:

1. Data Scientist

Peran ini bertanggung jawab penuh dalam siklus awal data, mencakup *Gathering*, *Assessing*, hingga *Cleaning Data* secara manual untuk menghasilkan wawasan bisnis yang terukur.

- a. **Manipulasi dan Komputasi Numerik:** Menggunakan pustaka *pandas* secara intensif untuk *data wrangling* secara *end-to-end*, serta *numpy* untuk operasi dan transformasi numerik berbasis *array*.
- b. **Visualisasi dan Explanatory Analysis:** *matplotlib* dan *seaborn* digunakan untuk merancang visualisasi data. Setiap grafik yang dihasilkan menjadi landasan mutlak dalam penarikan kesimpulan analisis (*insight*).
- c. **Dashboarding:** *Streamlit* difungsikan untuk merangkum hasil *Exploratory Data Analysis* (EDA) menjadi sebuah *dashboard* interaktif yang komunikatif.
- d. **Persiapan Data Lanjut:** Membangun *Data Dictionary* dan memastikan struktur akhir *dataset* telah terbebas dari anomali maupun kebocoran data (*data leakage*) sebelum diserahkan ke tahap pemodelan.

2. AI Engineer

Peran ini mengambil alih *dataset* yang telah dibersihkan untuk mengembangkan, melatih, dan menyempurnakan model prediktif cerdas murni dari awal (*from scratch*).

- a. **Pra-pemrosesan Ekstra dan Penyeimbangan Kelas**
Memanfaatkan modul *preprocessing* dari *scikit-learn*. Untuk mengatasi ketimpangan label diabetes, algoritma dioptimalkan menggunakan teknik hibrida dari *imbalanced-learn* berupa kombinasi *SMOTE* dan *RandomUnderSampler*.
- b. **Pengembangan Model Baseline**
Melatih model prediktif berbasis pohon keputusan seperti *Random Forest* dan *XGBoost* sebagai arsitektur klasifikasi final maupun pembandingan performa.
- c. **Arsitektur Deep Learning**
Pengembangan jaringan saraf tiruan menggunakan kerangka kerja *TensorFlow* (*Functional API* atau *Model Subclassing*). Sesuai batasan sistem, *AI Engineer* wajib menyuntikkan komponen lanjutan kustom (seperti *Custom Layer*, *Custom Loss Function*, atau *Custom Callback*). Penggunaan model siap pakai (*TensorFlow Hub*, *AutoML* diskriminatif, atau API pihak ketiga) dilarang.

3. Full-Stack Developer

Peran ini bertugas mentransformasikan antarmuka visual (*mockup*) dan model AI yang telah diekspor menjadi aplikasi *web end-to-end* yang andal dan fungsional.

a. Pengembangan Back-End (RESTful API)

Membangun kerangka layanan *Back-End* menggunakan Express (Node.js). API dikembangkan mengikuti standar konvensi URL RESTful untuk menangani logika sistem, berinteraksi dengan *database* (untuk penyimpanan data jika diperlukan), dan mengintegrasikan model AI sebagai fitur prediksi utama tanpa menyebabkan masalah kestabilan memori atau *crash*.

b. Pengembangan Front-End (UI/UX)

Kode antarmuka dibangun secara manual tanpa menggunakan *Web Generator*. Aplikasi disusun dengan tata letak responsif menggunakan utilitas desain modern seperti Bootstrap atau Tailwind CSS.

c. Integrasi dan Jaringan

Menggunakan klien HTTP Axios untuk menjalankan *networking calls* asinkron antara *browser* pengguna dengan API. Seluruh aset *Front-End* dikompilasi secara efisien menggunakan *module bundler* seperti Vite atau Webpack.

d. Deployment

Mendistribusikan (*deploy*) aplikasi perangkat lunak yang telah rampung ke penyedia layanan komputasi awan yang dioptimalkan untuk performa modern, seperti Vercel, Netlify, atau GitHub Pages.

2.3 Alur Kerja Proyek

Alur kerja mengadaptasi standar CRISP-DM yang diintegrasikan dengan siklus rekayasa perangkat lunak modern (*Software Development Life Cycle*):

1. Business Understanding

Mendefinisikan pertanyaan bisnis yang dapat diukur dan menentukan satu solusi utama berupa deteksi dini melalui arsitektur berbasis *web*.

2. Data Wrangling (End-to-End)

- a. *Gathering Data*: Mengakuisisi *dataset* dari sumber publik. Tim tidak menggunakan *dataset* yang sudah siap pakai, melainkan mengambil data mentah untuk diproses secara independen.

- b. *Assessing Data*: Mengevaluasi anomali, kualitas, dan struktur integritas data awal.
- c. *Cleaning Data*: Melakukan pembersihan data manual secara ekstensif hingga menghasilkan format *dataset* akhir yang sepenuhnya siap masuk ke tahap pemodelan.

3. Exploratory Data Analysis (EDA)

Menggali *insight* melalui visualisasi data menggunakan bagan dan grafik. Setiap visualisasi wajib diiringi dengan *explanatory analysis* berformat *markdown* atau teks naratif. Tidak ada penarikan kesimpulan analitik yang diizinkan tanpa dukungan bukti visualisasi yang valid.

4. Modeling (Pemodelan AI/ML)

Melatih model *Deep Learning* (dengan komponen *Custom Layer/Loss/Callback*) dan XGBoost pada data yang telah dibersihkan. Model dilatih murni dari awal (*from scratch*) tanpa menggunakan API eksternal (seperti Gemini/ChatGPT API) maupun *AutoML* diskriminatif.

5. UI/UX dan Integrasi Perangkat Lunak

- a. Merancang *mockup* representasi visual aplikasi.
- b. Membangun *RESTful API* menggunakan Express untuk menjembatani komunikasi antara antarmuka pengguna dengan model ML yang telah diekspor.
- c. Mengembangkan *Front-End* responsif yang melakukan panggilan jaringan (*networking calls*) ke *endpoint* API secara efisien, memastikan aplikasi berjalan tanpa cacat atau *crash*.

6. Deployment

Mempublikasikan *dashboard* Streamlit untuk pelaporan *insight* data, serta men-*deploy* aplikasi web fungsional (*Front-End* dan *Back-End*) ke *server production* (seperti Netlify/Vercel) agar dapat diakses oleh *end-user*.

BAB III

EKSPLORASI DATA DAN PRA-PEMROSESAN

3.1 Penilaian Data (Assessing Data)

Tahap awal dalam proses *Data Wrangling* adalah melakukan penilaian data (*Assessing Data*) untuk mengevaluasi struktur, integritas, dan kualitas dari *Diabetes Health Indicators Dataset* (BRFSS 2015). Evaluasi ini krusial untuk mengidentifikasi anomali, inkonsistensi, atau masalah struktural yang dapat mendistorsi hasil analisis analitik maupun performa algoritma *Machine Learning* di tahap selanjutnya. Proses inspeksi data dilakukan melalui beberapa tahapan pengujian:

1. Profiling Struktur dan Tipe Data

Pemeriksaan awal menggunakan metode komputasi dasar (seperti `df.info()`) menunjukkan bahwa *dataset* mentah memiliki dimensi yang sangat besar, yaitu terdiri dari **253.680 baris observasi** dan **22 kolom fitur**. Seluruh variabel direpresentasikan dalam tipe data numerik (`float64`) dengan alokasi memori sekitar 42,6 MB. Meskipun bertipe numerik, sebagian besar fitur pada dasarnya merupakan representasi kategorikal atau biner (misalnya `HighBP`, `Smoker`, dan variabel target `Diabetes_012` yang direpresentasikan dengan nilai 0, 1, dan 2).

2. Pemeriksaan Kelengkapan Data (*Missing Values*)

Evaluasi kelengkapan data dilakukan untuk mendeteksi adanya sel yang kosong atau tidak bernilai (*Null/NaN*). Hasil agregasi menunjukkan bahwa *dataset* memiliki kualitas kelengkapan 100%, di mana tidak ditemukan satu pun data yang kosong (*0 missing values*) di seluruh 22 kolom fitur. Hal ini mengeliminasi kebutuhan untuk melakukan teknik imputasi (pengisian nilai kosong) pada tahap pembersihan nanti.

3. Deteksi Duplikasi Rekam Medis

Mengingat data bersumber dari survei berskala nasional (BRFSS), sangat penting untuk memastikan setiap observasi mewakili individu yang unik. Pengecekan duplikasi mengungkap keberadaan **23.899 baris data yang identik persis** di seluruh 22 kolom. Meskipun kombinasi jawaban survei yang sama dapat terjadi secara kebetulan pada data kategorikal, duplikasi identik dalam skala ini berpotensi menyebabkan algoritma *Machine Learning* mempelajari pola yang berulang secara bias (*overfitting*), sehingga harus ditandai untuk dihapus.

4. Analisis Statistik Deskriptif dan Identifikasi Pencilan (*Outliers*)

Pemeriksaan distribusi nilai setiap variabel dilakukan melalui ringkasan statistik deskriptif. Fokus utama ditujukan pada variabel yang bersifat numerik kontinu, khususnya Indeks Massa Tubuh (BMI).

- a. Berdasarkan nilai kuartil, populasi dalam *dataset* ini memiliki rata-rata BMI sebesar 28,38 (mendekati batas kelebihan berat badan) dengan nilai median 27,0.
- b. Namun, ditemukan anomali ekstrem (pencilan/*outlier*) di mana nilai minimum tercatat pada angka 12 (sangat rendah secara medis) dan **nilai maksimum mencapai 98.0**.
- c. Nilai BMI 98.0 sangat jauh dari kuartil ketiga ($Q3 = 31.0$) dan secara fisiologis dinilai sangat tidak representatif untuk diikutsertakan dalam populasi umum.

Untuk memvalidasi temuan ini secara matematis, digunakan metode *Interquartile Range* (IQR). Dari perhitungan jarak antar-kuartil tersebut, teridentifikasi bahwa ribuan observasi berada di luar ambang batas wajar distribusi statistik, sehingga mengonfirmasi keberadaan *outliers* yang perlu ditindaklanjuti.

Secara keseluruhan, hasil penilaian data (*Assessing Data*) menyimpulkan bahwa meskipun *dataset* utuh tanpa nilai kosong, terdapat isu duplikasi data yang masif dan pencilan ekstrem pada fitur BMI yang wajib ditangani pada tahap *Cleaning Data*.

3.2 Pembersihan Data (*Cleaning Data*)

Tahap pembersihan data bertujuan untuk merektifikasi anomali yang ditemukan pada fase penilaian, mengeliminasi observasi yang terdistorsi (*noise*), dan mengoptimalkan struktur *dataset* sebelum masuk ke tahap analisis analitik maupun pelatihan model algoritma. Proses ini mencakup tiga prosedur utama:

1. Penanganan Pencilan (*Outlier Handling*) pada Fitur BMI

Menindaklanjuti temuan adanya nilai ekstrem hingga 98.0 pada Indeks Massa Tubuh (BMI), diterapkan metode penyaringan berbasis statistik menggunakan *Interquartile Range* (IQR).

- a. Untuk menjaga variansi data yang masih relevan dan masuk akal secara klinis, rentang IQR dikalibrasi menggunakan pengali 1.15.
- b. Perhitungan ini menghasilkan batas bawah (*lower bound*) di angka 16.0 dan batas atas (*upper bound*) di angka 39.0.

- c. Seluruh observasi dengan nilai BMI di luar rentang tersebut diidentifikasi sebagai pencilan yang tidak representatif dan dihapus secara permanen. Proses penyaringan ini berhasil mengeliminasi sebanyak 13.937 baris data, menyusutkan volume dari 253.680 menjadi 239.743 baris.
2. **Penghapusan Duplikasi Data (*Deduplication*)**

Langkah krusial berikutnya adalah mitigasi terhadap 23.899 baris observasi yang teridentifikasi sebagai duplikat absolut. Mempertahankan duplikasi masif dalam *dataset* berpotensi membuat algoritma *Machine Learning* menghafal pola yang repetitif sehingga memicu bias prediksi (*overfitting*). Seluruh data duplikat ini dihapus seutuhnya untuk menjamin bahwa setiap baris observasi mewakili individu yang unik.
 3. **Transformasi Tipe Data (*Data Type Casting*)**

Secara bawaan (*default*), seluruh variabel dalam *dataset* diimpor dengan tipe numerik desimal presisi tinggi (*float64*). Untuk meningkatkan efisiensi alokasi memori secara signifikan dan memastikan representasi logis data yang akurat, dilakukan penyesuaian (*casting*) tipe data secara spesifik:

 - a. **Konversi Boolean**

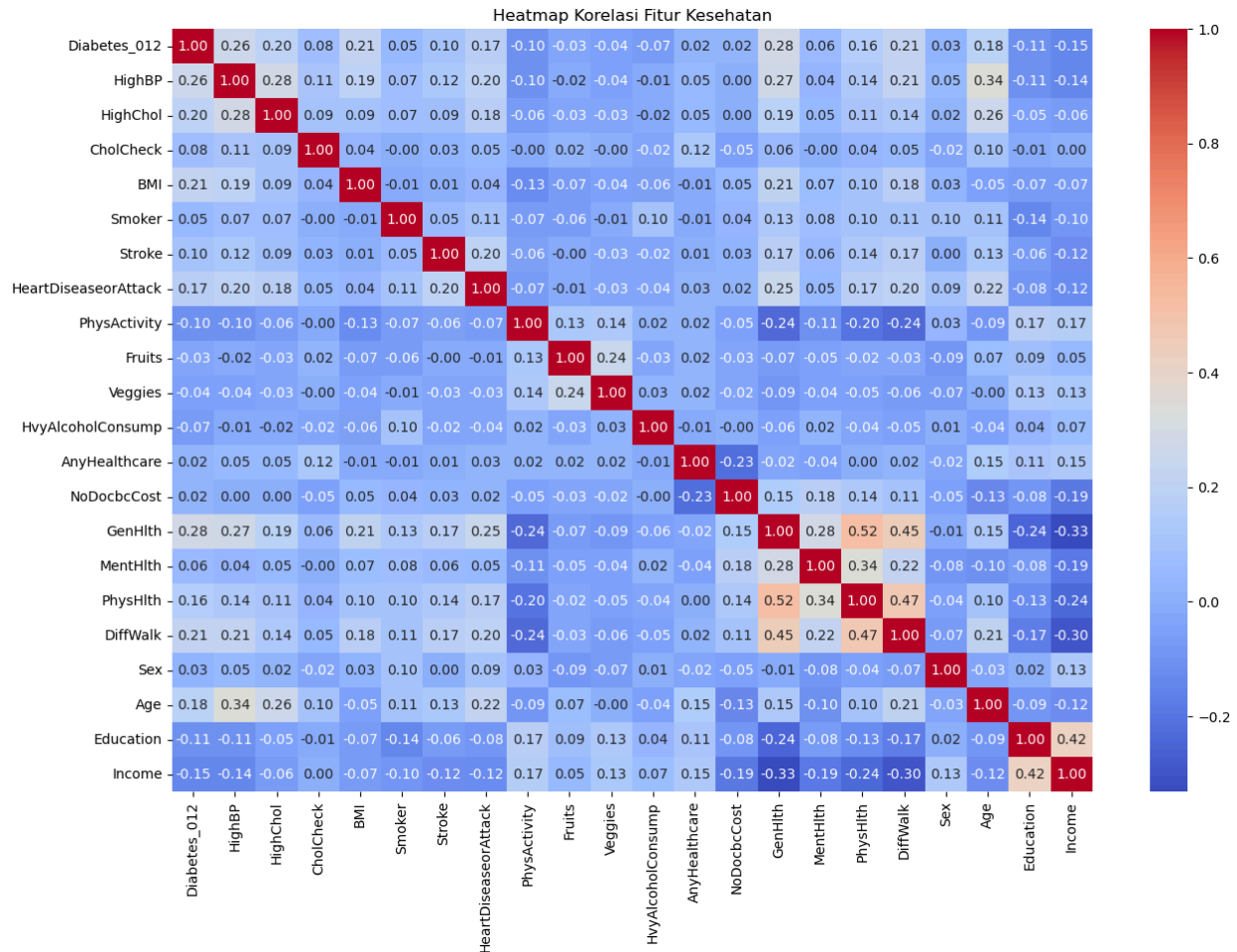
Terdapat 13 fitur prediktor yang pada dasarnya mendefinisikan status biner (Ya/Tidak), seperti *HighBP*, *HighChol*, *Smoker*, *PhysActivity*, dan indikator medis lainnya. Keseluruhan fitur ini dikonversi secara eksplisit menjadi tipe bool (*True/False*).
 - b. **Konversi Ordinal (*Ordered Categorical*)**

Fitur-fitur yang memiliki hierarki atau tingkatan logis tidak dibiarkan sebagai numerik biasa. Fitur seperti Education (6 tingkat pendidikan), Income (8 level pendapatan), Age (13 kategori usia), GenHlth (5 skala kesehatan umum), serta variabel target Diabetes_012 diubah menjadi tipe *categorical* berurutan (*ordered*). Transformasi ini sangat krusial agar sistem analitik dapat mengenali dan menghitung bobot gradasi hierarkis dari masing-masing fitur.

Melalui serangkaian tahapan rekayasa kualitas ini, diperoleh bentuk *dataset* bersih yang sangat ringan dalam memproses komputasi memori, memiliki integritas struktural tinggi, dan siap diumpankan ke dalam proses *Exploratory Data Analysis* (EDA).

3.3 Analisis Data Eksploratif (EDA) dan Insight Bisnis

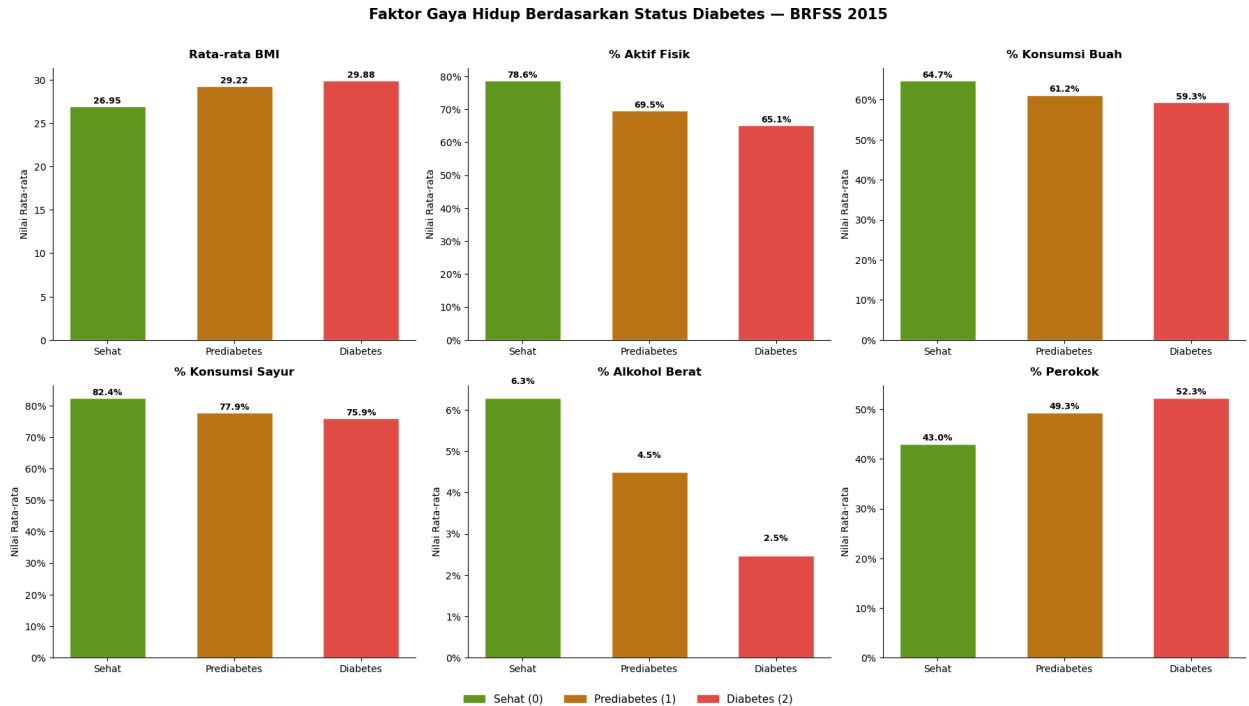
Tahap Analisis Data Eksploratif (EDA) berfokus pada pembedahan relasi antar-variabel untuk menjawab tiga rumusan masalah bisnis yang telah ditetapkan. Eksplorasi ini divalidasi melalui komputasi statistik dan visualisasi data guna menghasilkan wawasan (*insight*) bisnis yang dapat diukur.



Gambar 1. Correlation Heatmap

1. Evaluasi Pengaruh Faktor Gaya Hidup

Analisis pertama bertujuan mengidentifikasi metrik gaya hidup yang paling signifikan dalam membedakan antara kelompok sehat, prediabetes, dan penderita diabetes.



Gambar 2. Komparasi Faktor Gaya Hidup

a. Indeks Massa Tubuh (BMI)

BMI terbukti menjadi faktor diferensiator yang paling kuat. Terdapat tren peningkatan yang linear, di mana kelompok sehat mencatat rata-rata BMI sebesar 26,95, kelompok prediabetes sebesar 29,22, dan kelompok diabetes memuncak di angka 29,88. Selisih sebesar +2,93 poin antara penderita diabetes dengan kelompok sehat menegaskan korelasi positif yang kuat antara akumulasi massa tubuh berlebih dengan risiko diabetes.

b. Aktivitas Fisik

Tingkat aktivitas fisik menunjukkan korelasi negatif yang sangat jelas. Kelompok sehat merupakan demografi yang paling aktif secara fisik (78,6%). Angka ini menurun secara bertahap pada kelompok prediabetes (69,5%) dan anjlok secara drastis menjadi 65,1% pada penderita diabetes, merepresentasikan selisih keaktifan sebesar -13,54%.

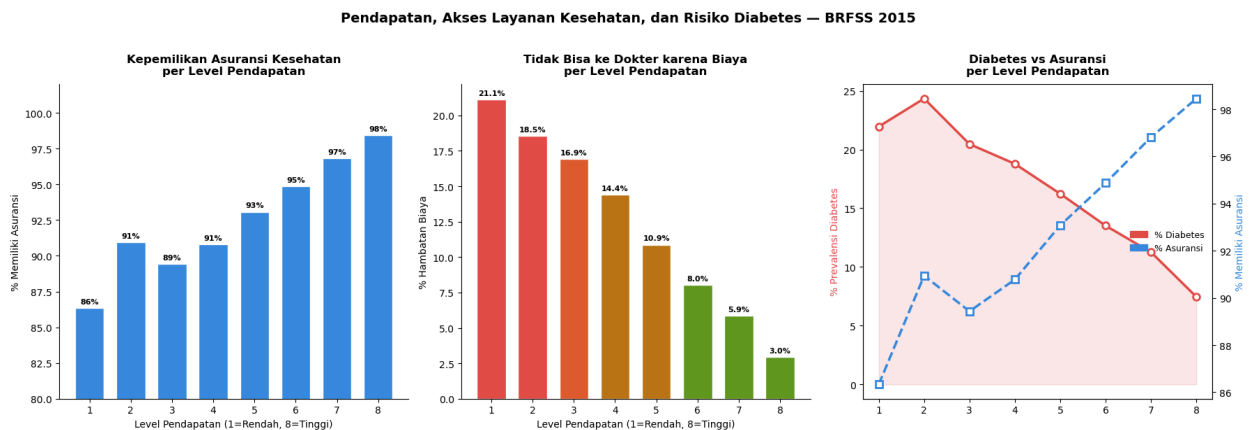
c. Perilaku Konsumsi (Diet dan Zat Adiktif)

Berbeda dengan asumsi umum, indikator diet seperti konsumsi buah dan sayur hanya memberikan dampak minor sebagai pembeda (penurunan marginal sekitar 5% hingga 6,5% pada penderita diabetes). Dari sisi konsumsi zat adiktif,

persentase perokok lebih tinggi pada penderita diabetes (+9,28%). Menariknya, porsi konsumsi alkohol berat justru mengalami penurunan sebesar -3,81% pada penderita diabetes dibandingkan kelompok sehat. Anomali ini mengindikasikan kemungkinan terjadinya modifikasi perilaku (*lifestyle modification*), di mana penderita cenderung mengurangi konsumsi alkohol setelah mendapatkan diagnosis medis.

2. Dampak Sosial-Ekonomi terhadap Akses Medis dan Prevalensi

Analisis kedua mengukur ketimpangan demografi sosial-ekonomi khususnya indikator pendapatan dan pendidikan serta kaitannya dengan risiko kesehatan.



Gambar 3. Komparasi Faktor Socio-ekonomi

a. Tingkat Pendapatan (*Income*)

Ditemukan korelasi yang berbanding terbalik secara tajam antara tingkat finansial dengan kemunculan penyakit. Kelompok populasi dengan tingkat pendapatan terendah (Level 1) mencatat angka prevalensi diabetes tertinggi, yakni sebesar 22,0%. Prevalensi ini terus menyusut seiring naiknya status ekonomi, hingga menyentuh angka 7,5% pada kelompok berpendapatan tertinggi (Level 8). Secara komparatif, risiko terjangkit diabetes pada kelompok ekonomi rentan hampir 2,9 kali lipat lebih besar.

b. Hambatan Biaya Medis

Tingginya prevalensi pada kelompok berpendapatan rendah selaras dengan tingginya hambatan mereka terhadap fasilitas medis. Sebanyak 21,1% individu dari kelas pendapatan terbawah melaporkan tidak mampu mengakses dokter

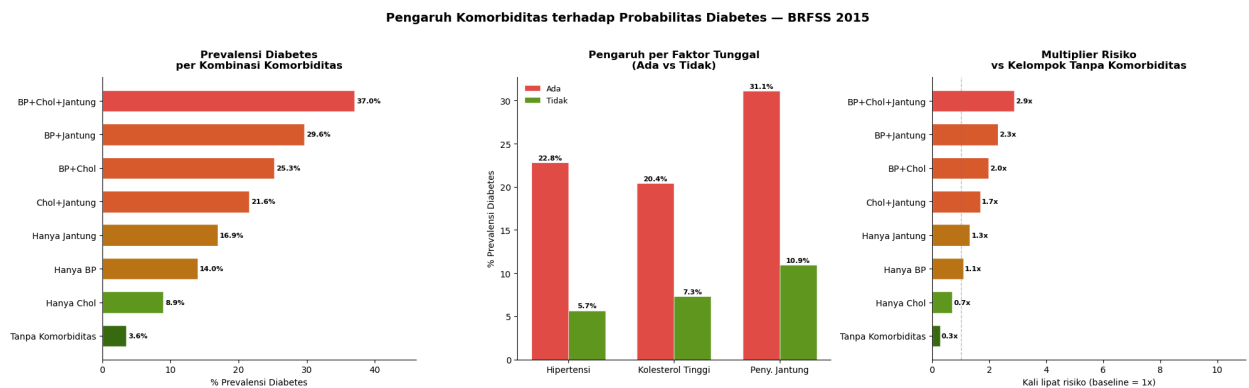
karena kendala finansial. Sebaliknya, tingkat kesulitan finansial medis ini hanya dilaporkan oleh 3,0% individu dari kelas pendapatan teratas.

c. **Tingkat Pendidikan (*Education*)**

Pola linier yang sama juga tercermin pada tingkat pendidikan. Individu yang berhenti pada jenjang pendidikan rendah memiliki tingkat prevalensi diabetes tertinggi di angka 28,0%. Angka risiko tersebut menurun secara persisten seiring dengan tingginya pencapaian akademik, hingga mencapai titik terendah sebesar 9,0% pada populasi berpendidikan tinggi.

3. **Efek Kumulatif Penyakit Penyerta (Komorbiditas)**

Analisis ketiga membedah bagaimana kombinasi komorbiditas kronis *HighBP*, *HighChol*, dan *HeartDiseaseorAttack* memengaruhi probabilitas diagnosis diabetes.



Gambar 4. Pengaruh Komorbiditas terhadap Probabilitas Diabetes

a. **Pengaruh Faktor Tunggal**

Ketika diisolasi sebagai faktor tunggal, riwayat penyakit jantung merupakan variabel komorbiditas pemicu tertinggi dengan peluang diabetes mencapai 31,13% (berada +20,19% di atas mereka yang tidak memiliki penyakit jantung). Faktor ini tercatat sedikit lebih dominan dibandingkan tekanan darah tinggi (22,79%) maupun kolesterol tinggi (20,41%).

b. **Efek Multiplikatif Kombinasi**

Probabilitas prevalensi dasar (*baseline*) bagi individu tanpa ketiga komorbiditas tersebut sangatlah rendah, yaitu hanya 3,6%. Namun, integrasi antar komorbiditas terbukti memicu efek multiplikatif yang agresif, melampaui perhitungan penjumlahan sederhana (aditif). Pada skenario terburuk, penderita yang memiliki kombinasi ketiga penyakit tersebut secara bersamaan dihadapkan pada

probabilitas diagnosis diabetes yang meroket hingga 37,0%. Angka ini merepresentasikan lonjakan risiko hingga lebih dari 10 kali lipat apabila dibandingkan langsung dengan kelompok *baseline* yang sepenuhnya sehat.

BAB IV

PENGEMBANGAN DAN PELATIHAN MODEL

4.1 Arsitektur Deep Learning (Model Subclassing dan Custom Layer)

Pengembangan model klasifikasi prediktif untuk risiko diabetes ini dibangun secara mandiri (*from scratch*) menggunakan kerangka kerja (*framework*) TensorFlow. Menghadapi kompleksitas data medis dan kebutuhan akan fleksibilitas eksperimental, arsitektur jaringan saraf tiruan (*Neural Network*) ini tidak dibangun menggunakan metode *Sequential* maupun *Functional API* standar. Sebagai gantinya, pendekatan **Model Subclassing** berbasis Pemrograman Berorientasi Objek (OOP) diimplementasikan untuk mendefinisikan perilaku arsitektur yang sepenuhnya kustom dan modular.

Secara struktural, arsitektur pemodelan ini terbagi menjadi dua komponen utama yang terdiri dari *Custom Layer* sebagai fondasi dasar, dan *Main Classifier*.

1. Desain Lapisan Kustom (*DenseBlock*)

Alih-alih menumpuk lapisan (*layers*) dasar secara berulang, sebuah komponen *Custom Layer* yang dinamakan *DenseBlock* diciptakan untuk mengenkapsulasi beberapa operasi komputasi matematika ke dalam satu blok terpadu. Blok ini mengadaptasi mekanisme *Pre-Activation* dengan urutan eksekusi sebagai berikut:

a. Transformasi Linear (*Dense Layer*)

Input diproses melalui lapisan *Dense* murni tanpa aktivasi awal (*activation=None*).

b. Normalisasi Batch (*Batch Normalization*)

Diaplikasikan secara langsung setelah transformasi linear dan sebelum fungsi aktivasi. Proses ini menstabilkan distribusi ruang fitur, mempercepat waktu konvergensi, dan mengurangi sensitivitas jaringan terhadap inisialisasi bobot awal.

c. Non-Linearitas (Aktivasi ReLU)

Fungsi *Rectified Linear Unit* (ReLU) diaplikasikan untuk memperkenalkan kapabilitas pemodelan non-linear agar jaringan mampu mempelajari pola data kesehatan yang kompleks.

d. **Regularisasi Dinamis (*Dropout*)**

Sebagian neuron dimatikan secara acak pada setiap iterasi pelatihan sesuai dengan *dropout rate* yang ditentukan. Hal ini memaksa jaringan menyebar bobot pembelajaran dan mencegah ketergantungan pada fitur tertentu, sehingga mereduksi fenomena *overfitting*.

e. **Koneksi Residual Opsional (*Residual Connection*)**

Terinspirasi dari arsitektur ResNet (*Residual Networks*), fungsi kondisional `use_residual` ditambahkan ke dalam blok. Ketika diaktifkan, input awal akan melewati lapisan proyeksi linier (tanpa bias) dan *Batch Normalization* tambahan sebelum ditambahkan dengan *output* utama blok. Koneksi *bypass* ini berfungsi sebagai jalan tol bagi gradien saat proses *backpropagation*, memecahkan masalah *vanishing gradient* (hilangnya gradien) pada lapisan yang dalam.

2. Arsitektur Jaringan Utama (*HealthClassifier*)

Model utama dibangun dengan mengorkestrasi empat DenseBlock yang disusun secara kaskade (mengecil atau menyerupai corong/ *funneling*). Desain corong ini bertujuan untuk mengekstraksi representasi fitur klinis yang sangat luas di awal, lalu menyaringnya menjadi representasi yang semakin spesifik dan padat di lapisan-lapisan akhir. Konfigurasi spesifik arsitekturnya adalah sebagai berikut:

a. **Input**

Menerima matriks berdimensi 9, merepresentasikan 9 fitur klinis prediktor (seperti *BMI*, *HighBP*, *Age_BMI_Risk*, dll).

b. **Block 1**

Berukuran masif dengan 512 unit neuron dan tingkat *Dropout* tertinggi (30%). Pada tahap ini, koneksi residual dinonaktifkan karena dimensi input (9) jauh lebih kecil dari dimensi *output* (512), dan jaringan masih berfokus pada ekstraksi fitur awal yang kasar.

c. **Block 2**

Menyusut menjadi 256 unit neuron dengan *Dropout* 20%. Koneksi residual (`use_residual=True`) mulai diaktifkan di sini untuk memastikan informasi fitur dasar tetap terjaga.

d. **Block 3**

Berukuran 128 unit neuron dengan *Dropout* 10% dan koneksi residual aktif.

e. **Block 4**

Blok ekstraksi fitur terakhir berukuran 64 unit dengan *Dropout* sangat kecil (5%). Koneksi residual dinonaktifkan untuk memfokuskan blok pada konvergensi representasi menuju klasifikasi.

f. **Output Layer**

Fitur yang telah diproses secara mendalam bermuara pada satu neuron *Dense* tunggal dengan fungsi aktivasi **Sigmoid**. Lapisan ini memetakan nilai komputasi ke dalam rentang kontinu 0,0 hingga 1,0 yang secara matematis merepresentasikan probabilitas risiko pasien mengidap diabetes.

Desain penurunan *Dropout Rate* secara gradual merupakan strategi yang disengaja. Lapisan awal (Block 1 dan 2) memiliki parameter yang sangat masif sehingga membutuhkan penalti regularisasi yang kuat. Sebaliknya, lapisan akhir (Block 4) telah menampung fitur yang sangat spesifik dan esensial, sehingga pemangkasan fitur secara acak (*dropout*) ditekan ke tingkat minimum agar tidak merusak representasi konseptual sebelum proses klasifikasi.

Secara keseluruhan, arsitektur *Deep Learning* kustom ini menghasilkan total **346.881 parameter**. Dari jumlah tersebut, sebanyak 344.193 parameter berpartisipasi aktif dalam pembaruan bobot selama pelatihan (*trainable*), sementara 2.688 parameter dikunci (*non-trainable*) sebagai variabel statistik internal untuk operasional *Batch Normalization*.

4.2 Komponen Kustom Lanjutan (Loss, Metric, dan Optimizer)

Dalam pengembangan model *Machine Learning* untuk diagnosis medis, tantangan terbesar yang sering dihadapi adalah ketidakseimbangan kelas (*class imbalance*) dan asimetri risiko. Meskipun proporsi data latih telah diseimbangkan secara eksternal menggunakan teknik SMOTE, kompleksitas ruang fitur kerap membuat model berisiko hanya mempelajari pola dari sampel yang mudah diklasifikasikan (*easy samples*) dan mengabaikan sampel perbatasan yang kritis (*hard samples*).

Untuk mengatasi limitasi arsitektur standar TensorFlow, tiga komponen komputasi kustom tingkat lanjut (*Custom Components*) diimplementasikan pada tahap kompilasi model:

1. Fungsi Kerugian Kustom (*Custom Loss Function*)

Fungsi kerugian bawaan seperti *Binary Crossentropy* (BCE) mengevaluasi seluruh kesalahan prediksi secara setara. Dalam konteks klasifikasi risiko diabetes, hal ini tidak ideal karena sampel yang mudah ditebak akan mendominasi nilai gradien dan menghentikan model untuk mempelajari kasus-kasus yang lebih samar.

Sebagai solusi, diimplementasikan kelas kustom *FocalLoss* yang mengadaptasi penelitian Lin et al. (2017). *Focal loss* memodifikasi BCE dengan menambahkan faktor modulasi, yang secara dinamis menyusutkan bobot kerugian dari sampel yang mudah diprediksi. Konfigurasi hiperparameter yang digunakan meliputi:

a. Parameter Fokus

Berfungsi sebagai eksponen pelemahan (*decay exponent*). Nilai 1,0 memastikan bahwa saat model sudah sangat yakin terhadap suatu prediksi (probabilitas mendekati 1 atau 0), kontribusi kerugian dari sampel tersebut ke pembaruan bobot akan direduksi secara signifikan. Hal ini memaksa jaringan untuk secara eksklusif memusatkan perhatian (*focusing*) pada klasifikasi sampel yang sulit (kasus klinis yang ambigu).

b. Faktor Penyeimbang

Karena ketimpangan kuantitas sampel telah diatasi sebelumnya via *resampling*, nilai diatur seimbang (0,5) guna mempertahankan stabilitas asimtotik tanpa mendiskriminasi kelas tertentu secara berlebihan.

2. Metrik Evaluasi Kustom (*Custom Metric*)

Metrik bawaan Keras seperti *Binary Accuracy* dinilai sangat menyesatkan (*misleading*) untuk kasus deteksi penyakit. Akurasi global yang tinggi bisa saja diraih hanya dengan memprediksi mayoritas pasien sebagai "Non-Diabetes", yang mana akan menghasilkan tingkat *False Negative* (pasien diabetes yang tidak terdeteksi) yang fatal secara klinis.

Oleh karena itu, kelas kustom *BinaryRecall* didefinisikan secara manual mewarisi *keras.metrics.Metric*. Metrik ini secara spesifik mengekstraksi nilai *True Positives* (TP) dan *False Negatives* (FN) pada level tensor di setiap iterasi *batch* pelatihan. Dengan mengintegrasikan metrik ini, evaluasi jaringan saraf tiruan diorientasikan murni pada optimalisasi sensitivitas (kemampuan model mendeteksi individu berisiko) di sepanjang siklus pelatihan.

3. Optimator dan Jadwal Laju Pembelajaran (*Optimizer dan Learning Rate Schedule*)

Proses pembaruan bobot (*weights update*) dilakukan melalui algoritma optimasi, namun laju pembelajarannya (*learning rate*) tidak dibiarkan statis. Untuk menavigasi lanskap kerugian (*loss landscape*) yang non-konveks secara optimal, digunakan strategi penjadwalan dinamis CosineDecayRestarts (*Stochastic Gradient Descent with Warm Restarts / SGDR*).

a. Pelemahan Kosinus (*Cosine Decay*)

Model memulai pelatihan dengan laju pembelajaran yang cukup agresif. Seiring berjalannya iterasi (*steps per epoch*), laju pembelajaran ini tidak diturunkan secara linear, melainkan melandai secara mulus mengikuti kurva fungsi kosinus. Hal ini memungkinkan pembaruan bobot yang sangat presisi ketika model mulai mendekati konvergensi global.

b. Siklus Restart Berkala

Jadwal ini dikonfigurasi untuk melakukan *restart* (mengembalikan laju pembelajaran ke titik awal) setiap 8 epok (*first_decay_steps*), dengan durasi siklus yang terus berlipat ganda (*t_mul=2.0*) dan puncak laju pembelajaran yang perlahan menyusut (*m_mul=0.9*). Mekanisme *restart* ini berfungsi sebagai mekanisme penyelamatan vital agar jaringan tidak terjebak secara permanen di dalam minimum lokal (*local minima*) akibat topologi data kesehatan yang rumit.

4.3 Skenario Pelatihan dan Custom Training Loop

Dalam alur kerja Keras konvensional, metode `model.fit()` sering kali menjadi standar untuk melatih model. Namun, untuk proyek klasifikasi risiko medis ini, fungsionalitas bawaan tersebut dinilai tidak memadai. Terdapat kebutuhan spesifik komputasi tingkat rendah, seperti perhitungan metrik *Validation Area Under the Curve* (Val AUC) secara global menggunakan *Scikit-Learn*, pencatatan nilai F2-Score per epok, dan integrasi modul pemantauan di level *Python* murni. Oleh karena itu, skenario pelatihan direkayasa ulang menggunakan **Custom Training Loop**.

Pelatihan ini didukung oleh *pipeline* data yang sangat efisien menggunakan `tf.data.Dataset`. Data disalurkan ke unit pemroses grafis (GPU/CPU) melalui mekanisme `.shuffle()`, `.batch()`, dan `.prefetch(tf.data.AUTOTUNE)`. Hal ini memastikan tidak ada *bottleneck* memori selama pelatihan berlangsung.

Siklus eksekusi di dalam *Custom Training Loop* dibagi menjadi empat fase utama pada setiap epok (*epoch*):

1. Fase Pelatihan (*Forward dan Backward Pass*)

Pada fase ini, komputasi dienkapsulasi di dalam blok `tf.GradientTape` untuk melacak seluruh operasi tensor.

- a. Model menerima asupan data per *batch* dan melakukan *forward pass* (`training=True`) untuk menghasilkan prediksi.
- b. Nilai kerugian (*loss*) dihitung menggunakan *Focal Loss*.
- c. Algoritma diferensiasi otomatis (*autodiff*) kemudian menghitung gradien kerugian terhadap seluruh parameter yang dapat dilatih (*trainable variables*).
- d. Gradien ini selanjutnya diaplikasikan oleh optimotor Adam untuk memperbarui bobot model (*backward pass*).
- e. Untuk memaksimalkan performa, fungsi pelatihan (`train_step`) dikompilasi menggunakan dekorator `@tf.function`, yang mengubah komputasi *eager execution* Python menjadi *TensorFlow Graph* statis yang jauh lebih cepat.

2. Fase Validasi Ekstensif

Setelah seluruh data latih dieksekusi, sistem berpindah ke fase validasi. Model melakukan *forward pass* pada data validasi dengan mode `training=False` (menonaktifkan *Dropout* dan membekukan status *Batch Normalization*). Berbeda dengan akurasi yang bisa dihitung per *batch*, metrik evaluasi medis (seperti AUC) memerlukan pandangan holistik terhadap seluruh distribusi data. Oleh karena itu, probabilitas prediksi dari seluruh *batch* validasi dikumpulkan secara agregat, kemudian fungsi `roc_auc_score` dari Scikit-Learn dieksekusi untuk mendapatkan nilai Val AUC yang sangat akurat.

3. Fase Pemantauan dan *Early Stopping* (Custom Callback)

Sistem dilengkapi dengan kelas pemantauan kustom (`TrainingCallback`) yang bertugas mengawasi jalannya pelatihan. Modul ini diinstruksikan untuk memonitor peningkatan **Val AUC**, bukan *Validation Loss*. Pada klasifikasi tidak seimbang (*imbalanced classification*), nilai *Val Loss* sering kali terus menyusut meskipun model gagal mengenali kelas minoritas. Sebaliknya, AUC merupakan representasi yang jauh lebih jujur karena mengukur separasi antara kelas positif (berisiko) dan negatif (sehat) di semua ambang batas.

Mekanisme *Early Stopping* diaplikasikan pada pemantauan ini dengan parameter batas kesabaran (*patience*) sebanyak 25 epok dan batas peningkatan minimum (*min_delta*) sebesar 0,001. Jika Val AUC tidak mengalami peningkatan melewati batas tersebut, sebuah penghitung iterasi (*wait counter*) akan bertambah.

4. Fase Pencatatan (*Logging ke TensorBoard*)

Di penghujung setiap epok, keseluruhan metrik termasuk *Loss*, *Accuracy*, *Recall*, Val AUC, perhitungan F2-Score, hingga dinamika fluktuasi *Learning Rate* akibat *Cosine Decay* dicatat dan diekspor ke dalam direktori log **TensorBoard**. Hal ini memungkinkan pemantauan visual metrik secara *real-time* dan mempermudah perbandingan performa antar eksperimen.

5. **Hasil Eksekusi Pelatihan** Berdasarkan parameter log pelatihan yang terekam, model menjalani iterasi pembelajaran yang sangat stabil di awal. Namun, setelah melewati epok pertengahan, performa Val AUC mulai stagnan. Mekanisme proteksi *Early Stopping* akhirnya terpicu dan menghentikan pelatihan secara paksa pada **Epok ke-49** karena tidak ada peningkatan performa lebih lanjut selama 25 epok berturut-turut.

Alih-alih menyimpan model pada kondisi akhir yang berpotensi mengalami *overfitting* ringan, sistem secara otomatis melakukan pemulihan bobot (*weights restoration*). Bobot model dikembalikan ke kondisi paling optimal yang tercatat pada **Epok ke-24**, yang mana sukses mencetak metrik performa tertinggi dengan **Best Val AUC sebesar 0,8228**.

4.4 Penyetelan Ambang Batas (*Threshold Tuning*)

Dalam algoritma klasifikasi biner standar, model secara otomatis akan menggunakan ambang batas probabilitas (*default threshold*) sebesar 0,5. Artinya, jika probabilitas *output* model, pasien diklasifikasikan sebagai "Berisiko" (Diabetes), dan sebaliknya. Namun, untuk aplikasi klinis dan deteksi dini penyakit, penggunaan ambang batas statis ini merupakan sebuah kecacatan desain (*design flaw*).

Dalam konteks skrining kesehatan, terdapat asimetri risiko yang sangat besar antara dua jenis kesalahan prediksi:

- a. **False Positive (Positif Palsu):** Pasien sehat diprediksi berisiko. Biaya dari kesalahan ini hanyalah kecemasan sementara dan biaya pemeriksaan laboratorium lanjutan yang pada akhirnya akan menunjukkan hasil normal.

- b. **False Negative (Negatif Palsu):** Pasien yang sebenarnya mengidap diabetes diprediksi sehat. Kesalahan ini sangat fatal karena pasien akan kehilangan kesempatan untuk mendapatkan intervensi dini, yang berujung pada komplikasi mematikan di masa depan.

Oleh karena itu, sistem ini membutuhkan penyetelan ambang batas secara presisi (*threshold tuning*) agar lebih agresif dalam mendeteksi risiko, dengan menoleransi peningkatan *False Positive* demi menekan *False Negative* hingga ke titik minimum.

1. Metrik Optimasi: F2-Score

Untuk mencari ambang batas yang paling ideal, optimasi tidak lagi menggunakan metrik Akurasi atau F1-Score (yang memberikan bobot setara pada *Precision* dan *Recall*). Proses ini menggunakan metrik **F2-Score**. Komputasi F2-Score memberikan bobot signifikansi pada *Recall* (kemampuan menemukan semua kasus positif) dua kali lipat lebih besar dibandingkan *Precision* (ketepatan dari prediksi positif). Metrik ini sangat selaras dengan kebutuhan medis di mana kita lebih memilih *over-diagnose* (mendeteksi berlebih) daripada melewatkan satu pasien yang benar-benar sakit.

2. Skenario *Threshold Sweep*

Proses pencarian titik potong optimal dilakukan secara otomatis menggunakan teknik *sweeping* pada himpunan data validasi (*Validation Set*). Probabilitas kontinu (0 hingga 1) dari *output* model dievaluasi secara berulang dengan menggeser ambang batas mulai dari **0,10 hingga 0,90** dengan interval kenaikan (*step*) sebesar **0,05**.

Pada setiap titik interval, sistem mengonversi probabilitas menjadi label biner sementara, lalu menghitung skor F2 secara makro.

- a. Pada *threshold* tinggi (misal: 0,70 - 0,90), model menjadi terlalu konservatif. *Precision* meningkat tajam, namun *Recall* hancur karena model menolak memprediksi "Berisiko" kecuali ia sangat yakin. Hal ini memicu jatuhnya F2-Score mendekati angka 0.
- b. Pada *threshold* rendah (misal: 0,10 - 0,30), model menjadi sangat sensitif. F2-Score melonjak tajam seiring dengan naiknya *Recall*. Hasil *sweeping* pada data validasi menunjukkan bahwa skor F2 teoretis tertinggi (0,8589) tercapai pada *threshold* ekstrem 0,30.

3. Kalibrasi Final (Threshold = 0,45)

Meskipun hasil komputasi *sweep* pada data validasi menunjuk angka 0,30 sebagai titik puncak F2-Score, implementasi *threshold* yang terlalu rendah di lingkungan dunia nyata akan menghasilkan tingkat *Precision* yang terlampau buruk. Jika sistem memberikan alarm palsu kepada hampir semua penggunanya, hal ini dapat menggerus kepercayaan pengguna (*user trust*) dan membebani kapasitas fasilitas kesehatan rujukan.

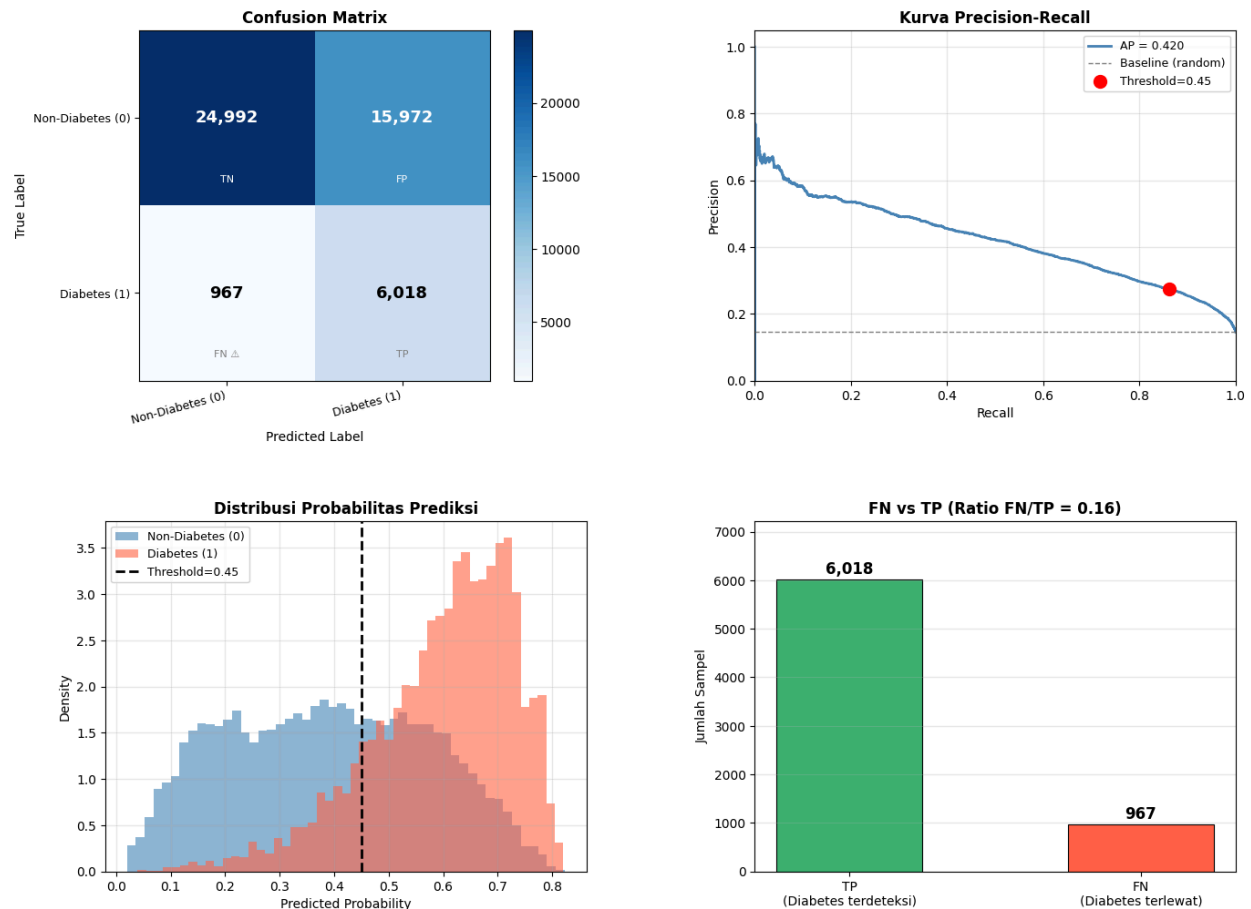
Sebagai kompromi klinis (*clinical trade-off*) yang mempertemukan agresivitas algoritma dengan rasionalitas operasional, ambang batas akhir dikalibrasi dan ditetapkan pada angka **0,45**. Nilai ini berada sedikit di bawah *default* 0,5 (tetap memprioritaskan *Recall*), namun cukup moderat untuk menyaring alarm palsu yang tidak perlu. Ambang batas 0,45 inilah yang kemudian dikunci secara permanen dan disuntikkan ke dalam *pipeline inference* untuk menguji performa final model pada data yang belum pernah dilihat sebelumnya (*Test Set*).

4.5 Evaluasi Final pada Data Uji (*Test Set*)

Setelah arsitektur *Deep Learning* kustom selesai dilatih dan ambang batas optimal ditentukan, pengujian pamungkas (*final inference*) dilakukan pada himpunan data uji (*Test Set*) independen. Himpunan ini terdiri dari 47.949 observasi klinis (40.964 sampel Non-Diabetes dan 6.985 sampel Diabetes) yang sama sekali tidak pernah dilihat oleh model selama fase pelatihan. Pada pengujian ini, probabilitas *output* dievaluasi menggunakan titik potong (*threshold*) final sebesar **0,45**, yang telah dikalibrasi secara khusus untuk memprioritaskan sensitivitas deteksi.

Hasil komputasi pada *Test Set* menunjukkan metrik performa yang sangat selaras dengan tujuan sistem skrining awal (*first-line screening*). Kinerja model dievaluasi dan divisualisasikan melalui beberapa indikator analitik berikut:

Evaluasi Model — Test Set



Gambar 5. Evaluasi Model Test

1. **Analisis *Confusion Matrix* dan Rasio Kritis** Pemetaan hasil prediksi ke dalam *Confusion Matrix* menghasilkan distribusi absolut sebagai berikut:

- True Negative (TN) = 24.992:** Pasien sehat yang diprediksi sehat dengan benar.
- True Positive (TP) = 6.018:** Pasien berisiko (diabetes) yang berhasil dideteksi dengan benar.
- False Positive (FP) = 15.972:** Pasien sehat yang keliru diprediksi sebagai berisiko (alarm palsu).
- False Negative (FN) = 967:** Pasien berisiko yang gagal dideteksi oleh model (terlewat).

Keberhasilan utama dari arsitektur model ini terletak pada kemampuannya mereduksi nilai *False Negative* hingga ke angka yang sangat rendah (967 kasus dari total 6.985

pasien positif). Evaluasi perbandingan antara pasien berisiko yang gagal ditangkap dengan yang berhasil dideteksi menghasilkan **Rasio FN/TP sebesar 0,16**. Angka ini merepresentasikan sebuah capaian klinis yang solid, membuktikan bahwa model mampu menangkap individu berisiko jauh lebih banyak (lebih dari 6 kali lipat) daripada jumlah pasien yang lolos dari deteksi. Tingginya angka *False Positive* (15.972) dinilai sebagai kompromi medis (*medical trade-off*) yang dapat diterima, mengingat biaya pencegahan lanjutan (seperti tes darah) jauh lebih rasional dibandingkan membiarkan penyakit tidak terdeteksi.

2. **Classification Report (Metrik Evaluasi Prediktif)**

Ditinjau dari metrik klasifikasi standar, model menunjukkan keandalan yang sangat tinggi dalam fungsi utamanya, yaitu peringatan dini:

- a. **Recall (Sensitivitas) Kelas Positif:** Model berhasil mencetak skor *Recall* sebesar **86%** untuk kelas Diabetes (Berisiko). Artinya, sistem kecerdasan buatan ini memiliki jaminan kapabilitas untuk mengenali 86% dari seluruh populasi pengidap diabetes.
- b. **Precision Kelas Positif:** Sebagai dampak matematis dari tingginya *Recall* dan pergeseran *threshold* ke angka 0,45, nilai *Precision* untuk kelas positif turun ke angka 27%. Secara operasional, ini berarti dari setiap 100 orang yang diprediksi "Berisiko" oleh sistem, sekitar 27 orang di antaranya adalah benar-benar pengidap diabetes.
- c. **Performa Global:** Secara keseluruhan, sistem mempertahankan tingkat akurasi global sebesar 65% dengan skor makro F1 berada di angka 0,58. Konfigurasi metrik ini sangat ideal untuk memosisikan aplikasi bukan sebagai alat vonis diagnosis absolut, melainkan sebagai instrumen triase preventif.

3. **Analisis Kurva dan Distribusi Probabilitas**

Evaluasi metrik diskrit juga divalidasi melalui observasi visual pada distribusi probabilitas prediksi berkelanjutan:

a. **Kurva Precision-Recall (PR Curve)**

Tidak seperti kurva ROC yang sering kali bias pada data tidak seimbang, Kurva PR menunjukkan hubungan yang lebih jujur antara sensitivitas dan ketepatan. Visualisasi ini menunjukkan *Average Precision* (AP) yang memadai di atas garis

baseline acak. Titik *threshold* 0,45 yang ditandai pada kurva menunjukkan letak posisi optimal yang menyelamatkan metrik *Recall* agar tidak jatuh secara eksponensial.

b. **Distribusi Probabilitas (Density Histogram)**

Pemetaan probabilitas menunjukkan adanya separasi (pemisahan) yang baik antara densitas pasien Non-Diabetes dan Diabetes. Histogram mengonfirmasi bahwa sebagian besar populasi kelas negatif (Non-Diabetes) mengelompok pada rentang probabilitas yang sangat rendah (mendekati 0,0), sementara kelas positif (Diabetes) memiliki distribusi tebal yang melewati batas vertikal *threshold* 0,45. Hal ini membuktikan bahwa lapisan-lapisan *DenseBlock* pada model berhasil mempelajari representasi yang membedakan kedua demografi klinis tersebut dengan cukup tegas.

BAB V

IMPLEMENTASI WEB DAN DEPLOYMENT

5.1 Arsitektur Sistem Terdistribusi

Untuk mengadopsi model *Deep Learning* ke dalam lingkungan aplikasi yang dapat diakses oleh publik, arsitektur monolitik tradisional dinilai tidak memadai. Pemrosesan model kecerdasan buatan dan komputasi nilai SHAP (*Shapley Additive exPlanations*) membutuhkan alokasi sumber daya komputasi (*CPU/Memory bound*) yang intensif. Jika disatukan dengan logika bisnis utama, hal ini berisiko memblokir antrian proses (seperti pada arsitektur *event-loop* Node.js) dan menyebabkan aplikasi tidak responsif (*bottleneck*).

Sebagai solusi, sistem aplikasi DiaLens direkayasa menggunakan pola pendekatan layanan mikro (*Microservices*). Pendekatan ini menjamin pemisahan beban kerja (*separation of concerns*), skalabilitas yang independen, dan keamanan data yang lebih terisolasi. Sistem ini secara struktural dibagi menjadi tiga lapisan utama:

1. Lapisan Antarmuka Pengguna (*Front-End Client* - Next.js)

Lapisan presentasi (*Presentation Tier*) ini dibangun menggunakan kerangka kerja React/Next.js. Lapisan ini beroperasi di sisi peramban (*client-side*) dan bertanggung jawab penuh atas interaksi pengguna (*UI/UX*).

- a. **Tanggung Jawab** Mengumpulkan input metrik kesehatan pengguna, melakukan validasi tipe data di sisi klien (seperti mencegah input nilai BMI kosong atau negatif), serta memvisualisasikan hasil probabilitas, tingkat risiko, dan rekam jejak (*history*) ke dalam dasbor yang interaktif.
- b. **Isolasi Front-End** tidak memiliki akses langsung ke *database* maupun ke mesin prediksi AI. Seluruh komunikasi data secara eksklusif dijemput oleh *Backend Gateway* melalui protokol HTTP.

2. Lapisan Gateway dan Orkestrasi Bisnis (*Backend* - Express.js)

Lapisan logika bisnis (*Logic Tier*) ini dibangun menggunakan Node.js dan Express.js. Lapisan ini bertindak sebagai peladen utama sekaligus *API Gateway* yang mengorkestrasi seluruh lalu lintas data di dalam sistem.

a. Manajemen Sesi dan Keamanan

Menangani autentikasi pengguna menggunakan *JSON Web Token* (JWT), memvalidasi hak akses, serta memproses alur registrasi dan *login*.

b. **Normalisasi Kontrak Data (*Contract Mapping*)**

Bertindak sebagai penerjemah antara *Front-End* dan mesin AI. *Front-End* mengirimkan data menggunakan konvensi *camelCase* standar pengembangan web, namun Express.js akan merakit ulang (*buildAiPayload*) dan menormalisasi *payload* tersebut menjadi konvensi spesifik yang diwajibkan oleh mesin AI serta memberikan perlindungan terhadap nilai *falsy*.

c. **Preservasi *Database***

Terhubung langsung ke *database* NoSQL (MongoDB) untuk mencatat aktivitas (*HealthRecord*). Metadata tambahan seperti tinggi (*heightCm*) dan berat badan (*weightKg*) disimpan di lapisan ini tanpa diteruskan ke mesin AI agar tidak merusak format matriks prediksi.

3. **Lapisan Inferensi Kecerdasan Buatan (*AI Inference Engine* - FastAPI)**

Lapisan ini merupakan layanan mikro (*microservice*) independen yang direkayasa khusus menggunakan kerangka kerja **FastAPI** (Python). Lapisan ini sepenuhnya terisolasi dari basis data pengguna maupun antarmuka web, menjadikannya sangat murni secara operasional.

a. **Inisialisasi Memori**

Saat peladen (*server*) FastAPI dinyalakan, sistem secara otomatis memuat tiga komponen esensial *Data Science* ke dalam memori RAM bobot model akhir (*best_weights.weights.h5*), skalar data prapemrosesan (*scaler_inference.pkl*), dan modul interpretasi (*shap_explainer.pkl*).

b. **Komputasi *Stateless***

Layanan ini menerima matriks 9 fitur klinis murni dari Express.js, melakukan *forward pass* pada model *Deep Learning*, mengeksekusi *GradientExplainer* dari SHAP, dan langsung mengembalikan paket data berupa nilai probabilitas (0-1), level risiko, dan 3 faktor pendorong utama (*top risk factors*). Pendekatan *stateless* ini memastikan *AI Engine* dapat ditingkatkan skalanya (*scale-out*) di masa depan tanpa memengaruhi logika *backend* utama.

Alur komunikasi dari ketiga lapisan ini menciptakan ekosistem terdistribusi yang sangat kokoh. Proses input dari pengguna (*Next.js*) akan divalidasi dan disimpan oleh *Backend* (*Express.js*), yang kemudian mendelegasikan tugas komputasi berat secara asinkron ke *AI*

Engine (FastAPI), sebelum akhirnya hasil tersebut dikembalikan dan ditampilkan ke hadapan pengguna.

5.2 Pengembangan Backend Gateway dan Manajemen Kontrak (Express.js)

Dalam arsitektur *microservices* DiaLens, peladen Node.js yang dibekali kerangka kerja **Express.js** beroperasi sebagai *API Gateway* sekaligus pengendali logika bisnis utama (*Main Controller*). Lapisan ini mengorkestrasi lalu lintas HTTP antara antarmuka pengguna (*Front-End*), pangkalan data (MongoDB), dan mesin inferensi AI (FastAPI).

Pengembangan pada lapisan *backend* ini tidak sekadar meneruskan data (*pass-through*), melainkan menerapkan perlindungan integritas data tingkat lanjut, yang terbagi ke dalam empat modul fungsionalitas utama:

1. Autentikasi dan Manajemen Sesi (JWT)

Untuk membatasi akses pada rekam jejak medis dan pengujian prediksi, sistem mengimplementasikan protokol keamanan berbasis *JSON Web Token* (JWT).

- a. Rute dasar (*/login* dan */register*) menghasilkan token akses (*access token*) dan token penyegar (*refresh token*).
- b. Guna menjaga kelancaran pengalaman pengguna (*User Experience*) tanpa memaksa autentikasi ulang yang repetitif saat token utama kedaluwarsa, sebuah *controller* dan rute khusus (*POST /api/health/refresh*) diregistrasikan. Rute ini bertugas memvalidasi *refresh token* dari klien dan secara dinamis merotasi sesi JWT secara asinkron.

2. Normalisasi *Payload* dan Manajemen Kontrak (*Contract Mapping*)

Tantangan rekayasa terbesar dalam arsitektur lintas layanan ini adalah ketidakcocokan kontrak data (*data contract mismatch*) antara *Front-End* dan *AI Engine*. Antarmuka pengguna mengirimkan parameter dengan konvensi *camelCase/lowercase* (misal: *highBP*, *bmi*), sedangkan mesin AI mewajibkan konvensi *PascalCase/Uppercase* secara ketat (misal: *HighBP*, *BMI*).

Untuk mencegah kegagalan inferensi, *backend* mengimplementasikan modul utilitas **buildAiPayload**. Modul ini bertugas melakukan translasi dan rekayasa *payload* dengan tingkat presisi tinggi:

a. Isolasi Fitur AI

Modul ini secara ketat menyaring (*filtering*) objek masukan dari *Front-End* dan hanya meneruskan 9 fitur esensial ke mesin AI.

b. Mitigasi Bug Falsy (Nullish Coalescing)

Pada iterasi awal, evaluasi nilai menggunakan operator logika menyebabkan input bernilai 0 (seperti HighBP: 0) dianggap sebagai nilai *falsy*, sehingga sistem salah memberikan nilai *default*. Hal ini sangat berbahaya secara klinis. Bug ini direktifikasi menggunakan operator *Nullish Coalescing* yang dipadukan dengan utilitas `toNumberOrDefault`. Pendekatan ini menjamin bahwa angka 0 tetap dipertahankan sebagai integer valid (bukan *null* atau untaian kosong).

3. Preservasi Metadata dan Penyimpanan Berbasis Skema (MongoDB)

Sistem wajib merekam riwayat pemeriksaan (*HealthRecord*) secara persisten menggunakan MongoDB melalui ODM (*Object Data Modeling*). Rekaman ini mencakup tiga skema kategori data: clinical, lifestyle, dan biometrics. Meskipun mesin AI dibatasi pada 9 fitur spesifik, arsitektur *backend* didesain secara fleksibel untuk menampung *metadata* tambahan dari *Front-End*, seperti tinggi badan (`heightCm`) dan berat badan (`weightKg`). *Metadata* ini ditangkap, divalidasi, dan disimpan ke dalam skema *database* internal secara permanen untuk kebutuhan visualisasi di riwayat (*history*), namun secara eksplisit di- *drop* (dibuang) dari objek *payload* yang dikirimkan melalui jaringan menuju *endpoint* FastAPI agar tidak melanggar aturan dimensi input model.

4. Validasi dan Kalibrasi Keluaran (Output Normalization)

Setelah *API Gateway* menerima respons balik dari mesin AI, sistem tidak langsung meneruskannya ke klien. Diperlukan tahap kalibrasi akhir (*post-processing*) untuk menjamin konsistensi tipe data:

a. Kalibrasi Tipe Data Probabilitas

Keluaran probabilitas dari FastAPI berpotensi berwujud *string* atau nilai persentase bulat (55). *Backend* mengeksekusi validasi menggunakan `Number.isFinite()` untuk mem-*parsing* nilai tersebut, dilanjutkan dengan operasi matematis pembagian bersyarat. Hal ini memaksa probabilitas untuk selalu berada pada skala pecahan kontinu **0,0 hingga 1,0**.

b. Perbaikan *Fallback Predictor*

Dalam skenario *edge-case* di mana mesin AI gagal merespons (*timeout*), *backend* menyediakan logika prediktor statis berbasis aturan (*rule-based fallback*). Logika ini dikalibrasi ulang agar memperhitungkan variabel usia (*Age*) berdasarkan indeks kategori yang benar (kategori 1–13 sesuai survei BRFSS), dan bukan memprosesnya sebagai umur aktual manusia.

c. Manajemen Respons Records

Data probabilitas hasil kalibrasi kemudian dibungkus dan disimpan ke dalam *database* dengan parameter *diabetesRisk*. Selanjutnya, format respons keluaran (*response body*) diseragamkan pembungkusnya (*res.data*) untuk diumpankan kembali kepada lapisan antarmuka pengguna.

5.3 Implementasi *Front-End* dan Antarmuka Pengguna (Next.js)

Lapisan presentasi dan interaksi pengguna dibangun menggunakan kerangka kerja **Next.js** (berbasis *App Router*). Pengembangan pada sisi klien (*client-side*) ini difokuskan pada penyediaan antarmuka (*UI/UX*) yang responsif sekaligus memastikan integritas aliran data menuju *Backend Gateway*. Terdapat empat aspek rekayasa utama yang diimplementasikan pada lapisan *Front-End* ini:

1. Manajemen Kontrak Data Ganda (*Dual Contract Management*)

Sistem *Front-End* berinteraksi dengan dua *endpoint* utama di *Backend* yang memiliki spesifikasi struktur (*casing*) yang berbeda. Untuk menjaga konsistensi, *Front-End* menerapkan penanganan kontrak data ganda secara ketat:

a. Fase Prediksi (POST */predict*)

Saat pengguna mengirimkan form pemeriksaan, *payload* dibentuk secara eksplisit menggunakan format *Uppercase/PascalCase* dan dipastikan bertipe numerik (contoh: HighBP, GenHlth, BMI). Format ini menjamin keselarasan dengan kontrak yang diwajibkan oleh mesin AI.

b. Fase Riwayat (GET */records*)

Saat merender halaman Dasbor (*Dashboard*) dan Riwayat (*History*), sistem membaca kembalian data dari pangkalan data yang berformat *camelCase* (contoh:

highBP, bmi, diabetesRisk). *Front-End* tidak memaksa translasi terbalik, melainkan membangun utilitas normalisasi (*defensive coding*) yang memprioritaskan pembacaan *camelCase*, dengan *Uppercase* hanya dipertahankan sebagai cadangan statis darurat (*fallback*).

2. Validasi Input dan Perlindungan Integritas AI

Untuk mencegah masuknya data tidak penting (*garbage-in, garbage-out*) yang dapat mendistorsi analisis *Deep Learning* dan interpretasi SHAP, halaman pemeriksaan (*Check*) dilengkapi dengan lapis validasi data yang agresif:

a. Pencegahan Nilai *Dummy*

Sistem memblokir pengiriman formulir jika input fisik (tinggi atau berat badan) bernilai 0, negatif, atau kosong. Bug potensial di mana aplikasi secara diam-diam mengirimkan nilai BMI cadangan (seperti 22.0) saat input tidak valid telah direktifikasi. *Payload* hanya dikirim apabila komputasi *Body Mass Index* (BMI) menghasilkan angka riil (*finite number*).

b. Manajemen Metadata Fisik

Metadata sekunder seperti tinggi badan (*heightCm*) dan berat badan (*weightKg*) ikut disertakan dalam pengiriman (*request*) untuk keperluan rekam medis *Backend*. Namun, formulir di sisi klien memastikan bahwa metadata ini diinkapsulasi sedemikian rupa agar tidak merusak 9 parameter wajib AI yang berada di akar *payload*. Selain itu, kalkulasi manipulatif (seperti menebak berat badan dari BMI jika data asli kosong) dihapus dari antarmuka riwayat untuk menjaga otentisitas rekam medis.

3. Visualisasi Dasbor dan Penentuan Tingkat Risiko

Komponen visual pada Dasbor dirancang untuk menerjemahkan angka probabilitas komputasi menjadi informasi klinis yang mudah dipahami oleh pengguna awam tanpa menimbulkan kebingungan:

a. Normalisasi Tipe Data Visual: Data riwayat medis yang ditarik dari *backend* sering kali memiliki tipe data campuran (angka dan untaian teks/*string*). Fungsi utilitas *normalizeYesNo* diterapkan untuk mem- *parsing* variasi nilai (seperti 1, '1', true, 'Ya') menjadi label antarmuka tunggal yang konsisten (Ya/Tidak) pada visualisasi metrik kolesterol dan hipertensi.

- b. **Resolusi Konflik Label Risiko:** Untuk mencegah kontradiksi informasi di antarmuka (misalnya: sistem menampilkan "Bukan Diabetes" sekaligus berstatus "Risiko Sedang"), penentuan label indikator dikalibrasi ulang. Status risiko mutlak diekstraksi berdasarkan rentang persentase parameter diabetesRisk (skala 0,0 - 1,0) yang digabungkan dengan analisis risk_level (Rendah, Sedang, Tinggi), alih-alih hanya bergantung pada keluaran biner prediksi semata.

4. Manajemen Status, Sesi, dan Penanganan Galat (*Error Handling*)

Kestabilan interaksi klien dan peladen dijaga melalui standarisasi utilitas jaringan dan manajemen sesi peramban (*browser session*):

- a. **Pemusatan Konfigurasi URL:** Untuk memfasilitasi lingkungan pengembangan (*staging*) dan produksi (*production*) yang dinamis, semua pemanggilan API menggunakan variabel terpusat API_BASE_URL. Hal ini mencegah *bug routing* di mana formulir pendaftaran/ *login* secara tidak sengaja ke peladen produksi yang dikodekan secara kaku (*hardcoded*).
- b. **Pembersihan Sesi Total (*Session Purge*):** Modul *logout* dikonfigurasi untuk menjalankan pembersihan area penyimpanan lokal (*localStorage*) secara menyeluruh. Selain menghapus *token* akses, sistem secara eksplisit menghancurkan *refresh token*, nama pengguna, dan objek identitas lainnya demi mencegah kebocoran sesi saat berpindah akun di perangkat yang sama.
- c. **Pemetaan Pesan Galat Dinamis:** Respons kegagalan (*error*) dari *Backend* memiliki arsitektur JSON yang bervariasi. *Front-End* menerapkan fungsi pemetaan dinamis (*getApiErrorMessage*) yang menelusuri hierarki respons untuk menangkap pesan galat spesifik dan merendernya dengan rapi ke dalam antarmuka UI, mencegah terjadinya *crash* pada aplikasi.

5.4 Deployment dan Infrastruktur Cloud

Tahap terakhir dari siklus pengembangan perangkat lunak pada sistem aplikasi DiaLens adalah memigrasikan arsitektur terdistribusi dari lingkungan pengembangan lokal ke infrastruktur *cloud*. Proses penyebaran (*deployment*) ini dirancang untuk memastikan ketersediaan layanan yang tinggi (*high availability*), latensi yang rendah, dan pengamanan kredensial operasional yang ketat.

Secara infrastruktur, sistem ini disebar ke dalam beberapa ekosistem *cloud* yang disesuaikan dengan kebutuhan spesifik masing-masing lapisan (*tier*):

1. Lingkungan Inferensi Kecerdasan Buatan (FastAPI)

Pada fase eksperimentasi (di dalam *Jupyter Notebook*), layanan FastAPI diekspos secara sementara menggunakan terowongan jaringan lokal (*local tunneling*) via pyngrok. Untuk tahap produksi, layanan mikro ini dipublikasikan ke peladen komputasi awan yang dioptimalkan untuk mengeksekusi beban kerja *Machine Learning*. Lingkungan penyebaran ini dikonfigurasi secara khusus agar memiliki alokasi memori (RAM) yang memadai. Hal ini merupakan syarat mutlak, mengingat peladen harus memuat infrastruktur pustaka berat seperti TensorFlow, *scaler* prapemrosesan, dan mengeksekusi algoritma interpretasi SHAP secara *real-time* untuk setiap permintaan prediksi tanpa menyebabkan peladen kehabisan memori (*Out-of-Memory / OOM crash*).

2. Orkestrasi Backend Gateway (Railway PaaS)

Lapisan *Backend Gateway* berbasis Express.js di-*deploy* secara publik memanfaatkan layanan *Platform-as-a-Service* (PaaS) dari Railway (mengudara pada domain dialens-backend-production.up.railway.app). Pemilihan infrastruktur ini didasarkan pada kapabilitasnya dalam menangani *Continuous Integration / Continuous Deployment* (CI/CD) secara mulus. Platform ini terhubung langsung dengan repositori, sehingga setiap perbaikan (*bug fix*) atau penambahan fitur dapat langsung dikompilasi ulang secara otomatis. Selain itu, Railway menyediakan enkripsi tingkat proksi untuk menyuntikkan variabel lingkungan (*environment variables*) seperti kunci rahasia JWT (JWT_SECRET) dan parameter koneksi *database* memastikan data sensitif tidak pernah terekspos di dalam basis kode sumber.

3. Pangkalan Data Terkelola (MongoDB Atlas)

Seluruh data operasional aplikasi disentralisasi pada ekosistem MongoDB Atlas. Pendekatan *Database-as-a-Service* (DBaaS) berbasis awan ini digunakan untuk menyimpan seluruh metadata, metrik klinis, dan riwayat probabilitas risiko pengguna. Penggunaan basis data terkelola ini menjamin persistensi data dengan replikasi otomatis dan perlindungan *firewall* berlapis. Pengembang tidak perlu lagi mengelola infrastruktur perangkat keras *server database* secara mandiri, melainkan dapat berfokus penuh pada optimasi kueri dan keandalan logika bisnis.

4. Distribusi Antarmuka Pengguna (*Front-End Hosting*)

Lapisan interaksi pengguna yang dibangun dengan kerangka kerja Next.js didistribusikan melalui infrastruktur *serverless* terkelola yang dirancang khusus untuk ekosistem React. Platform penyebaran ini memanfaatkan jaringan pengiriman konten (*Content Delivery Network* / CDN) global untuk melakukan *caching* pada aset-aset statis (gambar, *stylesheet*, skrip UI) dan menyajikannya dari titik peladen terdekat dengan lokasi geografis pengguna. Hal ini memangkas waktu muat (*loading time*) halaman Dasbor dan Formulir Pemeriksaan secara drastis, serta menangani *routing* proksi HTTP secara aman ke peladen Railway.

5. Pengamanan Kredensial AI Generatif (Claude API)

Sistem DiaLens memiliki ketergantungan pada pihak ketiga, yakni Anthropic Claude API, untuk meracik rekomendasi gaya hidup personal berdasarkan ekstraksi nilai SHAP. Parameter keamanan tertinggi diterapkan pada fase *deployment* integrasi ini (`API_KEY`) tidak pernah disisipkan atau dikirim melalui kode *Front-End*. Pemanggilan API generatif murni dilakukan secara asinkron (via `httpx`) dari peladen internal dengan perlindungan lapis *header*. Isolasi infrastruktur ini krusial untuk mencegah serangan pencurian kunci (*API key hijacking*) yang dapat merugikan kuota penagihan operasional aplikasi.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan serangkaian proses pengembangan, analisis, dan pengujian pada proyek *DiaLens - Your AI Diabetes Risk Screener*, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Keandalan Model Prediktif

Pengembangan arsitektur *Deep Learning* kustom melalui pendekatan *Model Subclassing* dan *DenseBlock* telah berhasil menciptakan mesin prediktif yang andal untuk keperluan skrining awal. Pengimplementasian komponen lanjutan seperti *Focal Loss* dan penyetelan ambang batas (*threshold tuning*) pada angka 0,45 terbukti sukses mengoptimalkan sensitivitas deteksi medis. Hal ini dibuktikan dengan capaian skor *Recall* sebesar 86% pada pengujian akhir, yang secara krusial berhasil menekan angka *False Negative* (pasien berisiko yang gagal terdeteksi) hingga ke titik minimum.

2. Validasi Faktor Risiko

Proses Analisis Data Eksploratif (EDA) secara empiris memvalidasi bahwa metrik gaya hidup seperti akumulasi Indeks Massa Tubuh (BMI) berlebih dan minimnya aktivitas fisik memiliki korelasi kuat terhadap peningkatan probabilitas diabetes. Lebih lanjut, ketimpangan demografi sosial-ekonomi (tingkat pendidikan dan pendapatan rendah) serta efek multiplikatif dari penyakit penyerta (hipertensi, kolesterol tinggi, dan penyakit jantung) terbukti mampu melonjakkan risiko prevalensi pasien hingga melampaui 10 kali lipat.

3. Ketangguhan Arsitektur Sistem Terdistribusi

Proyek ini berhasil mentransformasikan model kecerdasan buatan menjadi platform digital yang inklusif. Pemisahan beban kerja antara *Front-End* (Next.js), *API Gateway* pengelola kontrak data (Express.js), dan *AI Inference Engine* (FastAPI) menjamin stabilitas sistem dari kendala *bottleneck* maupun kebocoran memori.

4. Implementasi Edukasi Berbasis XAI

Sistem berhasil menjembatani algoritma klinis melalui integrasi *Explainable AI* (SHAP) dan kecerdasan buatan generatif (Claude API). Pendekatan ini memungkinkan sistem untuk tidak sekadar menebak risiko, tetapi juga menyajikan edukasi dan rekomendasi

gaya hidup preventif yang dipersonalisasi secara *real-time* kepada masyarakat luas melalui penyebaran (*deployment*) infrastruktur *cloud* yang aman.

6.2 Saran

Meskipun sistem DiaLens telah mencapai target fungsional yang dirancang, terdapat beberapa aspek yang dapat diekspansi dan disempurnakan pada iterasi pengembangan di masa depan:

1. Pembaruan dan Lokalisasi Dataset

Mesin inferensi saat ini dilatih menggunakan parameter dari *Diabetes Health Indicators Dataset* yang merupakan turunan dari survei BRFSS tahun 2015. Untuk meningkatkan akurasi kontekstual, disarankan agar pembaruan model berikutnya mengakuisisi data klinis mutakhir yang lebih merepresentasikan karakteristik genetik dan demografi lokal pasien di Indonesia.

2. Integrasi Data Objektif (IoT Medis)

Mengingat komputasi risiko saat ini bergantung sepenuhnya pada mekanisme pelaporan mandiri (*self-reported*) melalui formulir terstruktur, sistem rentan terhadap bias atau kesalahan pengisian oleh pengguna. Pengembangan selanjutnya disarankan untuk membuka protokol integrasi dengan perangkat *Internet of Things* (IoT) kesehatan, seperti *smartwatch* atau timbangan digital cerdas, agar asupan data prediktor menjadi lebih objektif.

3. Peningkatan Skalabilitas dan Otomatisasi Infrastruktur

Ekosistem DiaLens telah disebar memanfaatkan layanan *Platform-as-a-Service* (PaaS) dari Railway dan infrastruktur *serverless* terkelola. Guna menghadapi lonjakan lalu lintas pengguna (*traffic*) pada skala nasional, disarankan untuk mengimplementasikan orkestrasi kontainer (seperti *Kubernetes*) serta mengadopsi lapisan pengujian terotomatisasi (*automated unit testing*) yang komprehensif di dalam alur eksekusi CI/CD.